

Scaling GUI Agents with Visual State Transitions

Xiangyan Liu¹ Kaixin Li¹ Haonan Wang¹ Biao Wu³ Meng Fang⁴
 Longxu Dou² Chao Du² Michael Qizhe Shieh^{1*} Tianyu Pang²

¹NUS ²Sea AI Lab ³UTS ⁴University of Liverpool

Abstract

We introduce State Transition Pretraining (STP) as a new scaling axis for GUI agents. During the STP stage, we continually pretrain a unified multimodal model on visual state transitions by jointly optimizing forward dynamics (predicting next states from current states and actions) and inverse dynamics (predicting actions from state changes). This optimization equips the model with better action-grounded visual representations and an internal world model of GUI dynamics. When subsequently fine-tuned on trajectories with task instructions, our STP-trained models consistently outperform baselines trained solely via direct trajectory fine-tuning across agent benchmarks in both desktop and mobile GUI scenarios (AgentNetBench, AndroidControl, and GUIOdyssey). Further empirical studies show that joint dynamics optimization yields stable improvements over single-objective training, and downstream performance scales steadily with the volume of transition data. Our code is publicly available at <https://github.com/xyliugo/gui-state-transition-pretraining>.

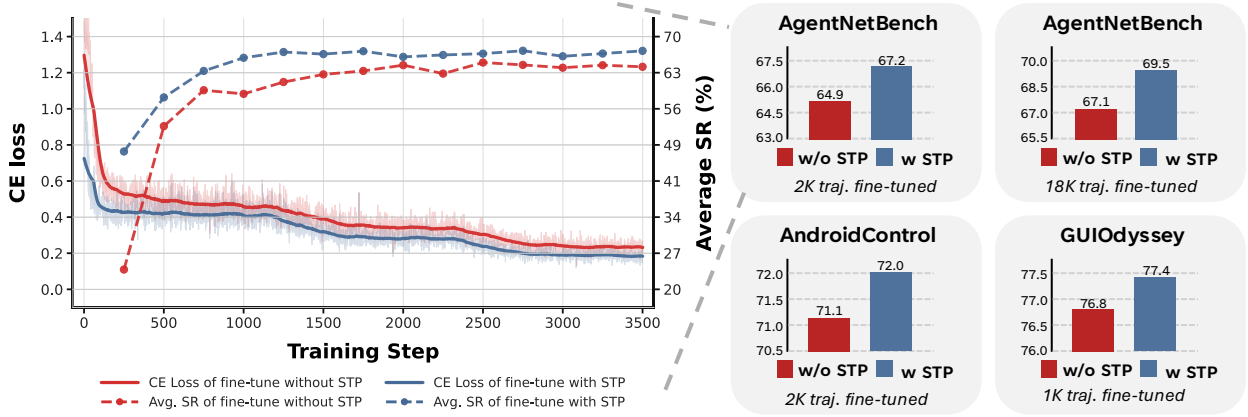


Figure 1: State Transition Pretraining (STP) improves both the training optimization and downstream performance of GUI agents. **Left:** Cross-entropy loss on AgentNet (training) and average success rate on AgentNetBench (evaluation) across fine-tuning steps. Both models are fine-tuned on the exact same 2K AgentNet Win&Mac trajectories. **Right:** Final performance across multiple benchmarks under different trajectory settings. Each subplot corresponds to a distinct training setup, and reports results under identical fine-tuning configurations with and without STP. Models are trained and evaluated on the corresponding dataset for each benchmark. The y-axis represents average success rate (%) for AgentNetBench, and step success rate (%) for AndroidControl and GUIOdyssey. These results demonstrate that the STP stage provides a strong initialization that complements trajectory fine-tuning.

*Correspondence to Michael Qizhe Shieh (michaelshieh@nus.edu.sg).

1 Introduction

Graphical user interfaces (GUIs) serve as the primary medium for human-software interaction (Myers, 1998). Recently, vision-language models have made it feasible to build GUI agents that automate complex workflows by interpreting task instructions and environment screenshots to predict executable actions (Bai et al., 2025a; Team et al., 2026a; Wang et al., 2025a;c; Liu et al., 2025; Tang et al., 2026a; Xu et al., 2026a). Yet, despite their growing practical importance (OpenAI, 2025; Anthropic, 2026; OpenAI, 2026), scaling GUI agents efficiently and effectively remains a significant challenge.

Reinforcement learning (RL) has recently emerged as a common approach for training GUI agents (Ye et al., 2025; Lai et al., 2025; Lu et al., 2025b; Shi et al., 2025; Xu et al., 2026a; Xue et al., 2026). However, scaling RL poses significant engineering challenges due to the need for complex interactive environments. As a result, trajectory fine-tuning using annotated demonstrations remains a primary approach (Yang et al., 2024; Xu et al., 2024; Qin et al., 2025; Yang et al., 2025), but it still faces limitations that restrict scalability. First, acquiring multi-step trajectory data is expensive because it typically requires manual labeling (Lu et al., 2025c; Wang et al., 2025c; Liu et al., 2025). Second, direct fine-tuning on these trajectories entangles visual grounding, dynamics modeling, and task planning into a single objective, making the optimization process difficult.

We therefore introduce **State Transition Pretraining (STP)** as a new *scaling axis* for GUI agents to complement trajectory fine-tuning. STP requires only step-level visual state transitions, which can be extracted from existing trajectories or collected automatically from interactive environments. During STP, each transition tuple (s_t, a_t, s_{t+1}) is used to instantiate two jointly optimized objectives. Specifically, ① **forward dynamics** operates as an image-text-to-image task that predicts the next state s_{t+1} from the current state and action (s_t, a_t) , enabling the model to develop an internal world model of GUI dynamics. ② **inverse dynamics** operates as an image-image-to-text task that predicts the action a_t from consecutive GUI screenshots (s_t, s_{t+1}) , encouraging the model to identify *where* the interaction occurred and to extract fine-grained, action-relevant visual features. This pretraining stage provides a better initialization for subsequent trajectory fine-tuning.

We evaluate the effectiveness of STP through extensive experiments on desktop and mobile GUI benchmarks. To naturally process and generate multimodal inputs and outputs, we adopt a unified multimodal model (Zhao et al., 2025) as the backbone. As shown in the main results in Table 1 and Table 2, when comparing models with and without STP under identical trajectory fine-tuning configurations, we find that STP consistently improves downstream performance across all settings. Specifically, on AgentNetBench (Wang et al., 2025c), STP increases the average success rate by 2.3% to 6.2% and the coordinate success rate by 1.6% to 6.8%, depending on the fine-tuning data split. In mobile environments, STP yields similar enhancements. On AndroidControl (Li et al., 2024), it improves the step success rate by 0.9% and the coordinate prediction accuracy by 1.3%. Similarly, on GUIOdyssey (Lu et al., 2025c), it increases the step success rate by 0.6% and the coordinate prediction accuracy by 1.6%. In addition, the left panel of Figure 1 illustrates how STP improves the optimization of trajectory fine-tuning as measured by cross-entropy loss.

Beyond these gains, STP exhibits strong *scalability* properties: scaling the transition pretraining data steadily improves downstream performance when the fine-tuning data is fixed. Moreover, STP continues to outperform fine-tune-only baselines as trajectory fine-tuning data scales from 2K to 18K under a fixed STP setup, suggesting that its gains come not only from additional data exposure but also from the structured learning signals provided by inverse and forward dynamics. Ablation studies further reveal that jointly optimizing inverse and forward dynamics provides complementary supervision, and that STP is almost equally effective whether transitions are sampled by trajectory or as individual steps. A detailed discussion of related work is provided in Section 4.

2 State Transition Pretraining for GUI Agents

We present **State Transition Pretraining (STP)**, a pretraining stage for GUI agents built on visual state transitions that require no additional task-level annotations. By modeling local state-action relationships prior to task-level trajectory optimization, it provides a better initialization and serves as a complementary scaling axis to subsequent trajectory fine-tuning.

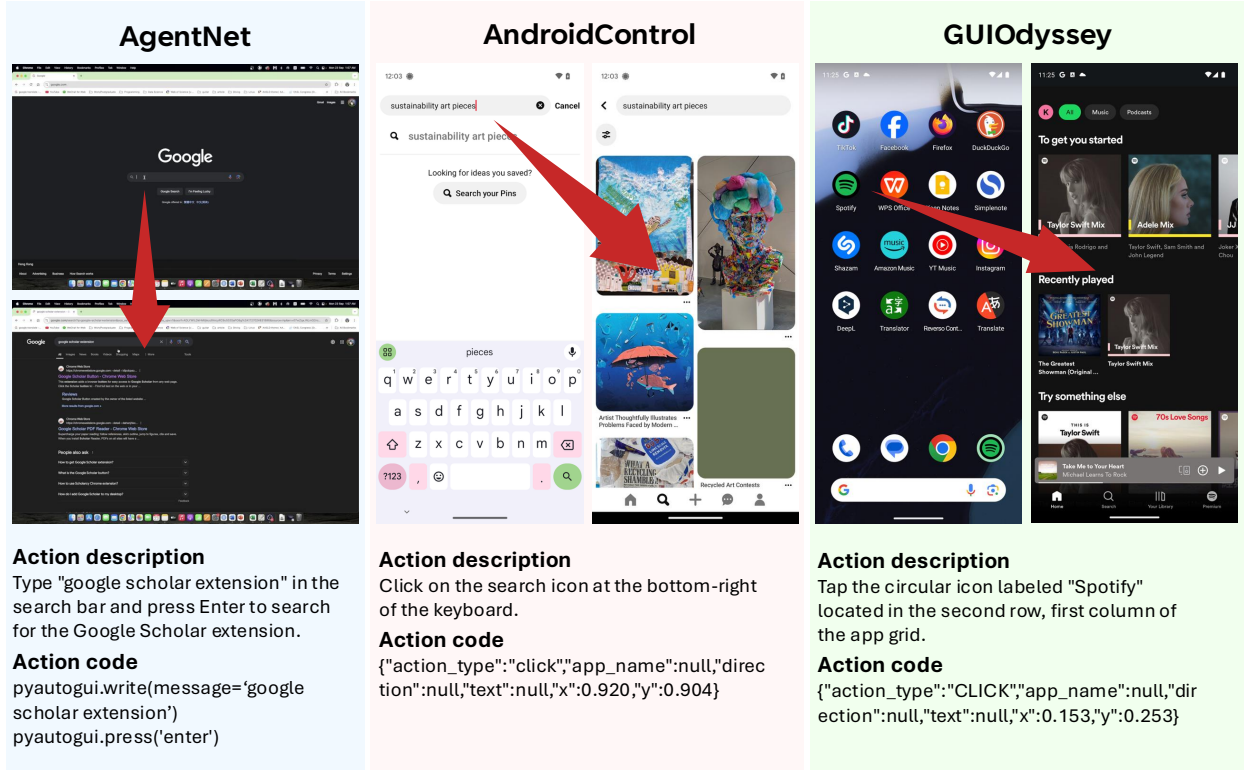


Figure 2: Examples of visual state transition data from three GUI control sources: AgentNet, AndroidControl, and GUIOdyssey. Each example consists of a transition tuple (s_t, a_t, s_{t+1}) , where s_t and s_{t+1} are two consecutive screenshots and a_t is the action executed between them. These transitions provide step-level supervision for STP without requiring task-level instruction annotations.

2.1 Background

Trajectory fine-tuning for GUI agents. We formulate GUI control as a sequential decision-making problem based on visual states. Given a task instruction u , at step t the agent observes the current GUI screenshot s_t as the state and the interaction history h_t , and predicts an action a_t with its specific parameters. In our setting, a_t is represented as a structured textual output that includes both a natural language description of the interaction and the corresponding executable GUI command (e.g., PyAutoGUI code), rather than solely a discrete action label. Here, the interaction history $h_t = \{(s_1, a_1), \dots, (s_{t-1}, a_{t-1})\}$ denotes the sequence of past state-action pairs prior to step t . This defines a conditional action distribution $p_\theta(a_t | u, s_t, h_t)$, which is optimized via supervised fine-tuning on expert trajectories:

$$\mathcal{L}_{\text{SFT}} = - \sum_t \log p_\theta(a_t | u, s_t, h_t). \quad (1)$$

However, scaling trajectory fine-tuning with task instructions faces two key challenges:

- **Data scarcity:** Collecting high-quality trajectories with accurate task instructions u is expensive and difficult to scale, as existing methods either rely heavily on human annotation (Li et al., 2024; Lu et al., 2025c; Wang et al., 2025c; Liu et al., 2025) or use purely synthetic approaches that inevitably introduce noise into the training data (He et al., 2025; Feizi et al., 2025).
- **Coupled learning objectives:** Eq. 1 implicitly couples multiple capabilities, including visual grounding (identifying and localizing elements in s_t), dynamics modeling (understanding action effects), and task

planning (following u), into a single next-action prediction objective, which can complicate optimization and reduce data efficiency (Cheng et al., 2024; Ni et al., 2025).

Unified multimodal models. Unified multimodal models (UMMs) (Xie et al., 2025; Chen et al., 2025b; Deng et al., 2025; Wu et al., 2025b) have recently gained increasing attention, as they handle flexible combinations of multimodal inputs and outputs within a single architecture. Specifically, a UMM can accept text, images, or their combination as input and produce text or images as output. This flexibility makes UMMs a natural backbone for state transition pretraining, as the two objectives correspond to native UMM task formats: inverse dynamics maps two screenshots to a textual action description $(s_t, s_{t+1}) \rightarrow a_t$, while forward dynamics maps a screenshot and an action to a new screenshot $(s_t, a_t) \rightarrow s_{t+1}$, enabling both to be learned jointly within a single model.

2.2 State Transition Data

While trajectory-level annotations are expensive, the step-level state transitions they contain are abundant and annotation-free. We decompose existing trajectory datasets into atomic tuples (s_t, a_t, s_{t+1}) , each consisting of two consecutive screenshots and the intervening action. Concretely, we extract transitions from AgentNet (Wang et al., 2025c) for desktop environments, and from AndroidControl (Li et al., 2024) and GUIOdyssey (Lu et al., 2025c) for mobile environments, as illustrated in Figure 2. Because these tuples capture only local state changes and discard task-level instructions, they remain reliable even when the original trajectory labels are noisy, yielding a high-quality transition corpus.

Given access to interactive environments, programmatic exploration and automated interaction offer a highly scalable method to expand transition collection beyond existing static datasets (Sun et al., 2025; Wu et al., 2026c). Our ablation in Figure 4 (right) shows that individually sampled step-level transitions are as effective as trajectory-level samples under the same transition budget, supporting the viability of this direction. While this presents a highly promising direction, we leave it to future work as it requires substantial engineering resources and falls outside the primary scope of our current study.

2.3 Pretraining GUI Agents with Inverse and Forward Dynamics

As shown in Figure 3, we continually pretrain the backbone UMM on visual state transitions prior to trajectory fine-tuning using two complementary objectives, inverse dynamics and forward dynamics:

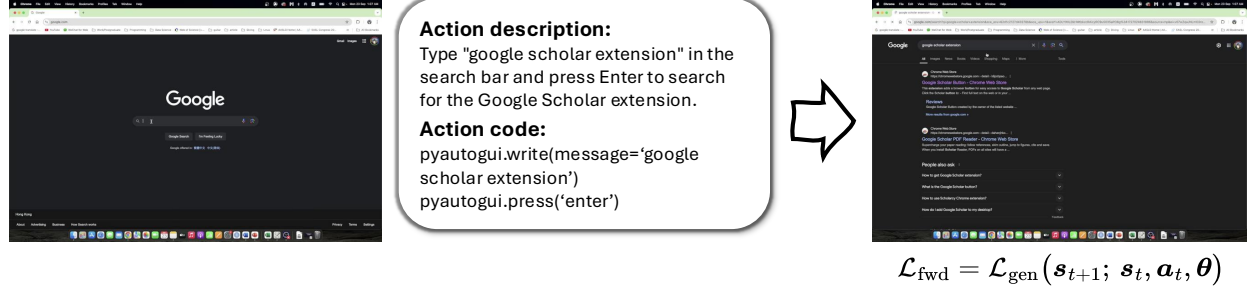
$$\mathcal{L}_{\text{inv}} = -\log p_{\theta}(a_t | s_t, s_{t+1}), \quad \mathcal{L}_{\text{fwd}} = \mathcal{L}_{\text{gen}}(s_{t+1}; s_t, a_t, \theta), \quad (2)$$

where $p_{\theta}(a_t | s_t, s_{t+1})$ denotes the standard next-token prediction probability for the action given consecutive states, and $\mathcal{L}_{\text{gen}}$ denotes the generative loss optimized by the UMM, which can be instantiated as an autoregressive (Chen et al., 2025b), diffusion-based (Zhou et al., 2025a), or flow matching (Ma et al., 2024; Deng et al., 2025) objective.

Forward dynamics. This objective requires the model to predict the next state s_{t+1} given the current state s_t and action a_t . Although the optimization is based on a generative loss, recent studies have shown that such generation implicitly provides supervision signals to the model’s perception and understanding components, improving their representations (Su et al., 2026; Wu et al., 2026b). By modeling *what changes* result from an action, the model captures the causal relationship between actions and visual state transitions (e.g., a dropdown list appearing after a click). This encourages the model to internalize how GUIs respond to actions, including changes in element states, content updates, and interface transitions, effectively forming an internal model of GUI dynamics.

Inverse dynamics. This objective requires the model to predict the action a_t given the state pair (s_t, s_{t+1}) . To do so, the model must identify the interaction location and the visual changes caused by the action, such as distinguishing a specific button from similar elements or detecting a text field update, while filtering out irrelevant details. This improves the encoder’s ability to extract fine-grained, action-relevant visual features, while also helping the language modeling component better interpret these representations into accurate action predictions.

Forward dynamics



Inverse dynamics

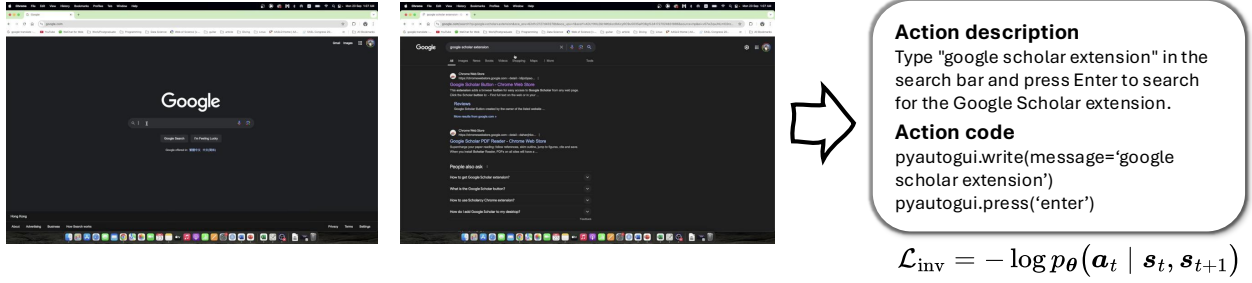


Figure 3: Illustration of inverse and forward dynamics in STP. Forward dynamics models the transition $(s_t, a_t) \rightarrow s_{t+1}$ by predicting the next state given the current state and action. Inverse dynamics models $(s_t, s_{t+1}) \rightarrow a_t$ by inferring the action that caused the transition.

Training protocol. We optimize the joint pretraining objective $\mathcal{L}_{\text{pre}} = \mathcal{L}_{\text{fwd}} + \lambda \mathcal{L}_{\text{inv}}$ on the transition corpus. By deliberately excluding task instructions, this stage enables the model to learn basic GUI capabilities directly from step-level supervision, without the additional complexity of task planning and instruction following. After STP, we perform trajectory fine-tuning to incorporate task-level supervision, allowing the model to integrate these capabilities with instruction-conditioned decision making.

3 Experiments

In this section, we evaluate State Transition Pretraining (STP) across multiple GUI scenarios, covering different data regimes and training configurations. In particular, we investigate the following aspects:

- **Robustness.** STP consistently improves downstream GUI control performance across both desktop and mobile scenarios, covering multiple datasets and benchmarks. (Table 1, Table 2)
- **Scalability.** Scaling transition data improves performance when fine-tuning data is fixed. Meanwhile, as the amount of fine-tuning data increases, models with STP continue to outperform those without it. (Table 1, Figure 6)
- **Impact of pretraining design.** The effectiveness of STP depends on the choice of pretraining objectives and which model components are updated. (Figure 5, Figure 6)

3.1 Experimental Setup

Backbone. We adopt BAGEL (Deng et al., 2025), a unified multimodal model (UMM) that natively supports both text and image generation. BAGEL uses a Mixture-of-Transformers (MoT) architecture containing two specific transformer experts for multimodal understanding and generation. These experts

share self-attention layers while maintaining separate parameters for their feed-forward networks. For visual processing, the architecture relies on a Vision Transformer (ViT) as the visual understanding encoder and a Variational Autoencoder (VAE) as the visual generation encoder and decoder. BAGEL uses a flow matching objective for image generation.

Because of this native structural support for both understanding and generation, BAGEL serves as a natural backbone for our two pretraining objectives: inverse dynamics (image + image $\rightarrow$ text) and forward dynamics (image + text $\rightarrow$ image). Operating with 7B active parameters per forward pass out of approximately 14B total, BAGEL offers a stronger and more stable foundation to isolate the effect of STP compared with other open-source UMMs such as Show-o (Xie et al., 2025) (1.3B), Harmon (Wu et al., 2025b) (1.5B), and OpenUni (Wu et al., 2025a) (3.6B).

Datasets and evaluation. We study two representative GUI scenarios: desktop and mobile. For the desktop scenario, we use **AgentNet** (Wang et al., 2025c) as the training corpus, which provides 18K Windows and macOS (Win&Mac) trajectories and 5K Ubuntu trajectories for subsequent trajectory fine-tuning. We further decompose the 18K Win&Mac trajectories into 320K state transitions for pretraining. We evaluate desktop performance on **AgentNetBench**,¹ which reports coordinate-, content-, function-level, and average success rates (SRs). For the mobile scenario, we use datasets from AndroidControl (Li et al., 2024) and GUIOdyssey (Lu et al., 2025c), which provide 12K and 7K trajectories, respectively. From these datasets, we extract 67K and 95K state transitions for pretraining. We measure model performance on AndroidControl² and GUIOdyssey using three commonly used metrics for GUI agents: action type prediction accuracy (Type), coordinate prediction accuracy (Grounding), and step success rate (SR). For all evaluations, we decode the model with temperature 0.6, sample four times per instance, and report the average.

Comparison setup of main experiments. Our main comparison is between direct trajectory fine-tuning (**FT w/o STP**, i.e., fine-tuning without state transition pretraining) and the same fine-tuning pipeline initialized with state transition pretraining (**FT w/ STP**). For the desktop scenario, we pretrain on 320K AgentNet state transitions and consider three fine-tuning settings: 2K Win&Mac trajectories, 5K Ubuntu trajectories, and 18K Win&Mac trajectories. For the mobile scenario, we pretrain on 67K AndroidControl state transitions and fine-tune on 3K AndroidControl trajectories; we also pretrain on 95K GUIOdyssey state transitions and fine-tune on 1K GUIOdyssey trajectories.

We additionally report the performance of several well-known models and checkpoints, including GPT-4o (OpenAI, 2024), OpenCUA-7B (Wang et al., 2025c), and UI-TARS-7B (Qin et al., 2025), in the main tables; however, these results are not directly comparable to our method due to differences in training data, backbone architectures, action spaces, and prompting setups across methods, and are therefore included *only for reference*. Our goal is not to claim state-of-the-art performance in absolute terms, but to isolate the effect of the STP stage through controlled comparisons that hold the backbone, fine-tuning data, and training configuration fixed. Furthermore, our method is largely orthogonal to most existing approaches, meaning it can be applied independently of their specific design choices.

Implementation details. All experiments are conducted on a single node with 8 GPUs, using either NVIDIA A100 40GB GPUs with CPU offloading or NVIDIA H20 96GB GPUs without offloading, depending on hardware availability. We use a learning rate of 1×10^{-5} and an EMA decay rate of 0.995 for both the state transition pretraining (STP) and trajectory fine-tuning stages. During STP, each training step consumes 8 inverse dynamics samples and 8 forward dynamics samples (one of each per GPU); for the joint pretraining objective in Section 2.3, we set $\lambda = 0.25$ for AgentNet and GUIOdyssey, and $\lambda = 0.5$ for AndroidControl. During trajectory fine-tuning, we train at step-level granularity following Eq. 1: on AgentNet and GUIOdyssey each GPU processes on average 3 samples per step (24 samples per step in total), and on AndroidControl each GPU processes on average 4 samples per step (32 samples per step in total). We use the same prompt template and input structure for training and inference: the model receives the full action history up to the current step together with only the current observation s_t , while past screenshots are excluded from h_t to reduce GPU memory usage. For training on AgentNet, we adopt the L1 (Action) template and do

¹AgentNetBench is a standalone benchmark built from Win&Mac samples.

²We evaluate exclusively on AndroidControl-High because our setting requires generating a low-level instruction before the executable action code (see Figure 2).

Model	Coordinate SR (%)	Content SR (%)	Function SR (%)	Average SR (%)
<i>Proprietary models</i>				
OpenAI CUA (OpenAI, 2025)	71.7	57.3	80.0	73.1
<i>Open-source models</i>				
Qwen2.5-VL-7B (Bai et al., 2025b)	50.7	40.8	3.1	48.0
Aguvis-7B (Xu et al., 2024)	56.7	43.3	0.0	52.4
Qwen2.5-VL-32B (Bai et al., 2025b)	66.6	47.2	41.5	64.8
Qwen2.5-VL-72B (Bai et al., 2025b)	67.2	52.6	50.5	67.0
OpenCUA-7B (Wang et al., 2025c)	79.0	62.0	44.3	75.2
OpenCUA-32B (Wang et al., 2025c)	81.9	66.1	55.7	79.1
Group 1a: STP on 320K AgentNet Win&Mac transitions + FT on 2K AgentNet Win&Mac trajectories				
FT w/o STP	65.4	49.5	65.0	64.9
FT w/ STP	68.0 $\uparrow$ 2.6	52.5 $\uparrow$ 3.0	69.1 $\uparrow$ 4.1	67.2 $\uparrow$ 2.3
Group 1b: STP on 320K AgentNet Win&Mac transitions + FT on 5K AgentNet Ubuntu trajectories				
FT w/o STP	54.2	50.2	75.3	56.9
FT w/ STP	61.0 $\uparrow$ 6.8	57.6 $\uparrow$ 7.4	73.2 $\downarrow$ 2.1	63.1 $\uparrow$ 6.2
Group 1c: STP on 320K AgentNet Win&Mac transitions + FT on 18K AgentNet Win&Mac trajectories				
FT w/o STP	69.4	49.3	62.9	67.1
FT w/ STP	71.0 $\uparrow$ 1.6	54.4 $\uparrow$ 5.1	65.0 $\uparrow$ 2.1	69.5 $\uparrow$ 2.4

Table 1: Main results on AgentNetBench under different fine-tuning regimes. We compare trajectory fine-tuning with and without STP across three settings with varying data scales and domains. STP is performed on 320K AgentNet Win&Mac state transitions. For FT w/ STP, the colored values indicate the difference relative to FT w/o STP, where red denotes improvement and blue denotes decline.

not use the original thought or observation annotations provided in the dataset. For the training of the forward objective, we remove the additional VAE conditioning input in BAGEL. We also note that VAE encoder features are excluded from the input context during training and inference, as they are not used in text-generation tasks (i.e., inverse dynamics and trajectory fine-tuning). All pretraining on state transitions is conducted for one epoch. For each fine-tuning run, we report the performance of the best-performing checkpoint on the corresponding benchmark, with the identical selection protocol applied to models with and without STP. Concrete examples of how trajectories are converted into fine-tuning and transition samples are provided in Appendix A. All other hyperparameters follow the default settings in the BAGEL codebase.³

3.2 Main Results

STP benefits subsequent trajectory fine-tuning across diverse GUI scenarios. Tables 1 and 2 demonstrate that, given the same fine-tuning data, models using STP outperform those without it across both desktop and mobile environments. On AgentNetBench (Group 1a), using only 2K Win&Mac trajectories for fine-tuning, the pretraining stage increases the average SR from 64.9% to 67.2% (+2.3%). Similarly, in mobile scenarios, SR increases from 71.1% to 72.0% (+0.9%) on AndroidControl (Group 2) and from 76.8% to 77.4% (+0.6%) on GUIOdyssey (Group 3). Grounding accuracy also sees improvements of +1.3% and +1.6% on AndroidControl and GUIOdyssey, respectively. These gains hold across diverse environments and evaluation metrics, supporting our core claim: STP provides an effective initialization strategy for later trajectory learning across various GUI scenarios.

STP helps under domain-shifted fine-tuning. In Group 1b, we pretrain on Win&Mac transitions but fine-tune on Ubuntu trajectories. This setup introduces a clear domain shift in both the operating system and application distribution, particularly because the target evaluation (AgentNetBench) is based on Win&Mac environments. Despite this mismatch during the fine-tuning stage, applying STP significantly increases the average SR from 56.9% to 63.1% (+6.2%). This result indicates that target-domain supervision can be effectively supplied through STP rather than relying entirely on trajectory fine-tuning. Consequently, STP offers a practical approach for adapting models to target GUI domains where full trajectory annotation is prohibitively expensive.

³<https://github.com/bytedance-seed/BAGEL>

Model	AndroidControl-High			GUIOdyssey		
	Type (%)	Grounding (%)	SR (%)	Type (%)	Grounding (%)	SR (%)
<i>Proprietary models</i>						
Claude-Computer-Use (Anthropic, 2024)	63.7	0.0	12.5	60.9	0.0	3.1
GPT-4o (OpenAI, 2024)	66.3	0.0	20.8	34.3	0.0	3.3
<i>Open-source models</i>						
SeeClick (Cheng et al., 2024)	82.9	62.9	59.1	71.0	52.4	53.9
OS-Atlas-7B (Wu et al., 2024)	85.2	78.5	71.2	84.5	67.8	62.0
UI-TARS-7B (Qin et al., 2025)	83.7	80.5	72.5	94.6	90.1	87.0
GUI-R1-7B (Luo et al., 2025b)	71.6	65.6	51.7	65.5	43.6	38.8
AgentCPM-GUI (Zhang et al., 2025b)	77.7	-	69.2	90.0	-	75.0
UI-Venus-Navi-7B (Gu et al., 2025)	86.5	-	76.1	87.3	-	71.5
UI-S1-7B (Lu et al., 2025d)	79.9	73.4	68.2	76.3	61.7	59.5
GUI-Shift-Mimo-SFT (Gao et al., 2025)	87.2	-	73.4	86.1	-	60.7
Group 2: STP on 67K AndroidControl transitions + FT on 3K AndroidControl trajectories						
FT w/o STP	86.3	68.2	71.1	-	-	-
FT w/ STP	87.1 $\uparrow 0.8$	69.5 $\uparrow 1.3$	72.0 $\uparrow 0.9$	-	-	-
Group 3: STP on 95K GUIOdyssey transitions + FT on 1K GUIOdyssey trajectories						
FT w/o STP	-	-	-	91.3	78.8	76.8
FT w/ STP	-	-	-	91.5 $\uparrow 0.2$	80.4 $\uparrow 1.6$	77.4 $\uparrow 0.6$

Table 2: Main results on AndroidControl and GUIOdyssey. For each benchmark, we compare trajectory fine-tuning with and without STP. For FT w/ STP, the colored values indicate the difference relative to FT w/o STP, where red denotes improvement and blue denotes decline.

STP yields improvements even without unseen-data exposure. When pretraining improves performance, it is often because the model is exposed to data not seen during the fine-tuning stage. To determine if our results simply come from this additional data exposure, we analyze Group 1c. In this configuration, the 320K state transitions used for STP are extracted directly from the 18K trajectories used for fine-tuning. As a result, the pretraining stage provides no new screenshots, actions, or environment interactions beyond those already used for fine-tuning. Despite this, STP still delivers a performance increase of +2.4%. This improvement is similar to Group 1a (+2.3%), where the fine-tuning data is restricted to 2K trajectories and the STP corpus introduces a large amount of additional data. This comparison suggests that the performance gains are not simply driven by exposure to unseen data patterns. Instead, they also arise from the structured learning signals provided by the inverse and forward dynamics objectives.

Smaller gains in mobile scenarios may stem from prior data exposure and lower task complexity. While STP improves performance across all evaluated settings, the gains on mobile benchmarks are relatively smaller than those on desktop. We attribute this to two primary factors. First, AndroidControl and GUIOdyssey were released before 2025, so the backbone model (BAGEL), released in May 2025, may have encountered related data during pretraining. This prior exposure reduces the marginal benefit of applying additional STP. Second, mobile tasks in these datasets are generally simpler. This is evident in the training dynamics: as shown in Figure 4 (left), the cross-entropy loss on mobile datasets decreases much faster than on the desktop dataset. Because the model fits the mobile data more easily, there is simply less performance headroom for STP to fill. These observations suggest that the marginal benefit of STP grows with task complexity and with the degree to which the target distribution is under-represented in the backbone’s original pretraining data, both of which imply that STP will remain valuable as GUI benchmarks continue to grow in difficulty and diversity.

3.3 Ablation Studies

Joint inverse and forward dynamics provide the most reliable pretraining signal. Figure 5 compares *inverse-only*, *forward-only*, and *joint inverse+forward* pretraining under different fine-tuning settings. Across all three settings, the joint objective consistently achieves the best performance over the fine-tune-only baseline. In contrast, the effectiveness of single-objective pretraining is less consistent. On the two desktop settings, both *inverse-only* and *forward-only* outperform the fine-tune-only baseline.

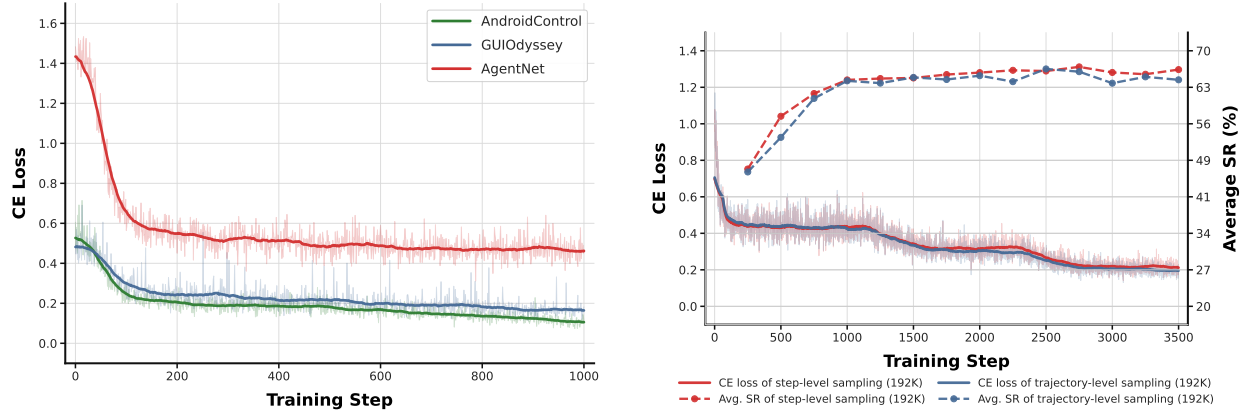


Figure 4: Training dynamics under different STP conditions. **Left:** Cross-entropy loss during fine-tuning across the three datasets used in our experiments; mobile datasets (AndroidControl, GUIOdyssey) converge substantially faster than the desktop dataset (AgentNet), indicating lower task complexity and less headroom for STP to fill. **Right:** Trajectory-level vs. step-level transition sampling for STP under a shared 192K-transition budget (fine-tuning on 2K trajectories); the two strategies yield comparable training dynamics and downstream performance.

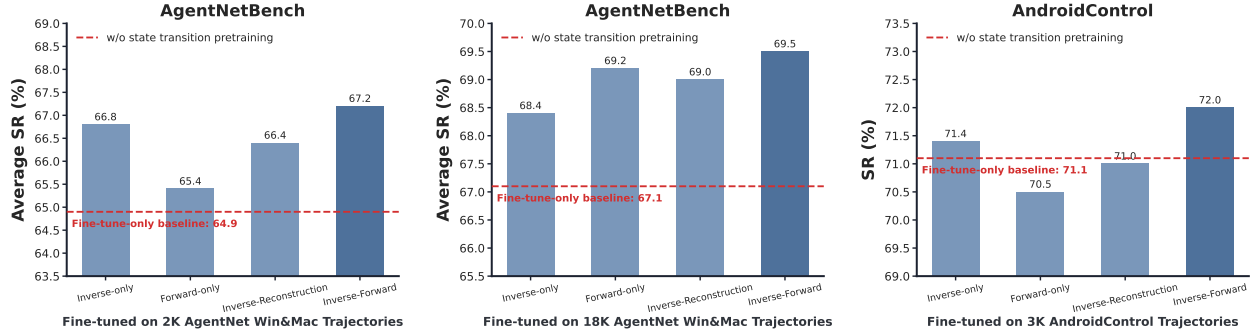


Figure 5: **Ablation on pretraining objectives.** We compare *inverse-only*, *forward-only*, *inverse+forward*, and *inverse+reconstruction* pretraining across different fine-tuning settings. Results are reported as average success rate on AgentNetBench and step accuracy on AndroidControl. Each bar corresponds to a pretraining objective, and the red dashed line indicates the fine-tune-only baseline without state transition pretraining.

However, this pattern does not hold on AndroidControl: *forward-only* drops by -0.6% , while *inverse-only* yields only a modest gain of $+0.3\%$. These results suggest that inverse and forward dynamics provide complementary training signals, and that optimizing them jointly yields a more robust pretraining strategy than relying on either objective alone.

Forward dynamics provides more action-aware supervision than reconstruction. As shown in Figure 5, *inverse+forward* consistently outperforms *inverse+reconstruction* across all settings. The key difference lies in the supervision signal: reconstruction is formulated as generating the same state from $(s_t, \text{“describe this image in detail”}) \rightarrow s_t$, which mainly reinforces static visual understanding and can be solved without modeling action effects. In contrast, forward dynamics learns the transition from (s_t, a_t) to s_{t+1} , explicitly capturing how actions change the UI state. When combined with inverse dynamics, this provides more *action-aware* supervision and leads to better alignment with downstream decision making, resulting in consistently stronger performance.

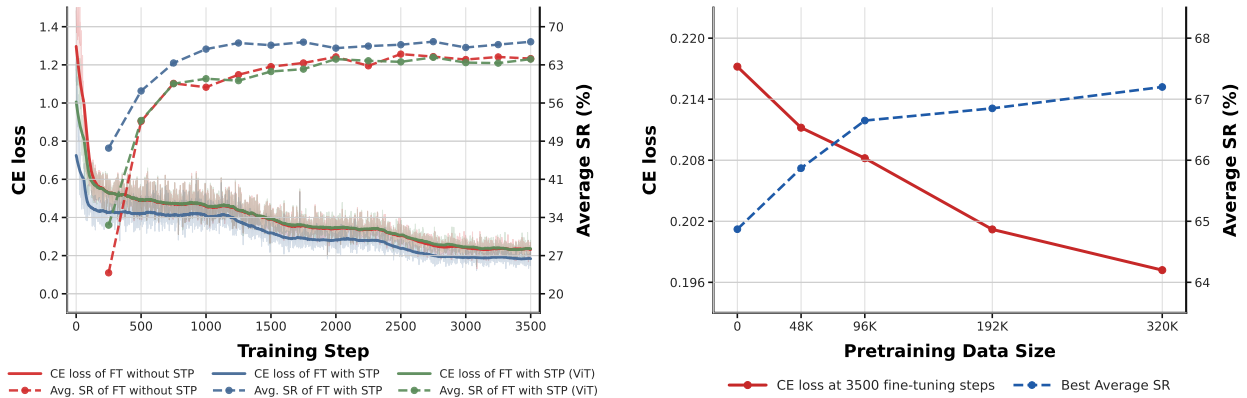


Figure 6: **Ablation on visual understanding encoder tuning and pretraining scale.** The left plot compares different pretraining strategies, including no STP, full-model STP, and STP with only the visual understanding encoder (ViT in BAGEL) being updated, with Cross-Entropy (CE) loss and average SR tracked over training steps. The right plot compares different pretraining data sizes, reporting CE loss after 3500 fine-tuning steps and the best average SR achieved under each setting.

Improving visual representations alone is insufficient for GUI control. Figure 6 (left) also compares two pretraining strategies: updating the entire model versus updating only the visual understanding encoder ⁴. While tuning only the visual understanding encoder brings a small improvement over no pretraining at the early stage of fine-tuning, the gains quickly diminish, as reflected in both cross-entropy loss and evaluation performance. In contrast, full-model tuning yields consistent improvements throughout training. This suggests that better visual representations from the ViT alone are not sufficient for strong GUI control. Instead, the language model component plays a critical role in interpreting visual features and producing structured actions, and must be jointly optimized with the visual encoder to fully benefit from STP.

Scaling state transition data leads to consistent performance gains. As shown in Figure 6 (right), increasing the amount of transition data used in STP steadily improves downstream performance on AgentNetBench, where the average SR consistently rises as the data scales. Additionally, for a fixed number of fine-tuning steps, models pretrained with more transition data achieve lower cross-entropy loss, indicating a better initialization for downstream optimization. These findings demonstrate that scaling transition data provides a reliable method for improving GUI agents. This supports our view of STP as an orthogonal scaling axis: larger transition corpora lead to better initialization even when the trajectory fine-tuning budget is fixed. In practice, this approach is highly advantageous because transition data can be obtained at a significantly lower cost than fully annotated trajectories.

Transition sampling granularity has minimal impact on STP effectiveness. We compare two transition sampling strategies under the same budget of 192K transitions: (1) *trajectory-level*, where we sample approximately 11K trajectories and decompose them into 192K transitions; and (2) *step-level*, where we directly sample 192K individual transitions from the full 320K AgentNet Win&Mac transition pool. As shown in Figure 4 (right), the two strategies yield comparable performance (66.85% vs. 66.45% average SR), despite step-level sampling covering greater data diversity since transitions within the same trajectory tend to share similar applications and visual contexts. This suggests that STP effectiveness is primarily driven by the volume of individual transitions rather than trajectory-level structure, and that future step-level data collected through automated exploration could serve as an effective source.

Practical cost of adding STP. A natural concern is whether the gains from STP justify the additional pretraining stage. In our largest configuration (320K AgentNet transitions), STP adds roughly one epoch of joint inverse and forward dynamics training on top of the standard trajectory fine-tuning pipeline, without introducing any additional labels. This cost is small relative to the trajectory-annotation effort that would

⁴The visual generation encoder and decoder (VAE components) are kept frozen in all settings.

otherwise be needed to collect a comparable amount of task-level supervision, and it is amortized across all downstream fine-tuning runs that start from the same STP checkpoint. Moreover, all trajectory fine-tuning runs are trained to convergence, as reflected by the plateaued cross-entropy loss in Figure 4 (left), so the gains from STP cannot be recovered by simply allocating more compute to extended fine-tuning. Combined with the observation that transitions can be extracted from the same trajectory pool used for fine-tuning (Group 1c), STP offers a favorable trade-off in practice: a one-time compute investment that yields consistent improvements across multiple downstream configurations without requiring any new human annotation.

4 Related Work

4.1 GUI Agents

GUI agents show promise as general interfaces for real-world digital tasks, driving interest in agents that perceive, reason, and act within desktop and mobile environments (Wang et al., 2025a; Zhou et al., 2025b; Team et al., 2026b; Lin et al., 2026; Cao et al., 2026a). Existing research covers specific-capability benchmarks (Hsiao et al., 2025; Liu et al., 2024; Cheng et al., 2024; Li et al., 2025; Mu et al., 2025; Cao et al., 2025) alongside realistic agentic benchmarks featuring longer horizons, richer observations, and complex multi-step interactions (Deng et al., 2023; Li et al., 2024; Rawles et al., 2024; Xie et al., 2024; Lu et al., 2025c; Wang et al., 2025c). To improve capability, prior work explores supervised fine-tuning (SFT) from expert demonstrations (Xu et al., 2024; Qin et al., 2025; Wang et al., 2025c; Liu et al., 2025) and reinforcement learning (RL) in interactive environments (Ye et al., 2025; Lai et al., 2025; Lu et al., 2025b; Shi et al., 2025; Gu et al., 2026; Yao et al., 2026; Liu et al., 2026). However, scaling these training methods presents significant challenges. RL requires complex environments and extensive engineering support (Wu et al., 2026a; Tang et al., 2026b; Kar et al., 2026), while SFT relies on multi-step trajectory annotations that are expensive to collect and difficult to scale (Gao et al., 2025; Wu et al., 2026c). We introduce State Transition Pretraining (STP) as a new scaling axis for GUI agents. By learning from isolated step-level state transitions that are significantly cheaper and highly scalable, STP provides a strong initialization for subsequent trajectory fine-tuning.

4.2 Learning from State Transitions

Recent work has increasingly studied state transitions as a useful learning signal for interactive digital environments. One line of work focuses on *world modeling* for web, mobile, and computer-use agents, where models predict future interface states, state differences, or executable environment representations to support lookahead reasoning and decision making (Chae et al., 2024; Luo et al., 2025a; Cao et al., 2026b; Zheng et al., 2026; Koh et al., 2026; Guan et al., 2026; Xu et al., 2026b; Bai et al., 2026), interface simulation (Rivard et al., 2025), and action refinement or safety checking (Shen et al., 2026; Yu et al., 2026). Most recently, Qwen-AgentWorld (Zuo et al., 2026) scales this idea into a language world model that simulates diverse agent environments over textual observations, showing that world-model training also serves as an effective warm-up for downstream agent policies. Another line studies *transition-based supervision* more directly, showing that state changes themselves can provide scalable training signals through self-supervised inverse dynamics (Gao et al., 2025), self-supervised forward dynamics or implicit world modeling (Wang et al., 2025b; FAIR CodeGen team et al., 2025; Zhang et al., 2025a; Chen et al., 2025a), textual foresight (Burns et al., 2024), or action recovery from passive videos (Lu et al., 2025a; Xiong et al., 2026). Together, these studies suggest that step-level transitions contain rich information about both action semantics and environment response.

However, existing approaches often treat forward and inverse dynamics separately or rely on indirect supervision. Specifically, UI-Oceanus (Wu et al., 2026c) demonstrates the scalability of forward dynamics learning but uses textual state differences rather than direct visual prediction as its objective. DeFI (Zhang et al., 2026) shows that forward and inverse dynamics provide complementary signals but trains them with separate models rather than jointly. Falcon-UI (Shen et al., 2024) also decouples instruction-free GUI pretraining from instruction tuning, but focuses on static single-screenshot understanding rather than state transitions. In contrast, we jointly optimize both objectives within a single unified multimodal model directly

over visual states, enabling the two directions to share representations and provide mutual supervision throughout pretraining.

5 Limitations

Our study has several limitations. First, most transition data used in our experiments is decomposed from existing trajectory datasets, so the scale and diversity of pretraining remain constrained by the available trajectories. While our step-level vs. trajectory-level ablation (Figure 4, right) suggests that individually collected transitions are equally effective, we have not yet demonstrated this at scales beyond existing trajectory corpora. Second, although our results indicate that step-level transitions are a promising supervision source, large-scale automated transition collection from interactive environments remains future work and requires substantial engineering effort, including reliable environment setup, action space design, and quality filtering of automatically collected transitions. Third, the forward dynamics objective introduces additional image-generation pretraining cost compared with text-only inverse dynamics, which practitioners may need to balance against the downstream gains, particularly under constrained compute budgets. Fourth, the full STP recipe assumes a backbone with native image-generation capability; on standard vision-language backbones without image generation, only the inverse dynamics objective is directly applicable, a restricted setting that prior work has already explored (Gao et al., 2025).

6 Broader Impacts

Improving GUI agents may make software more accessible and reduce the cost of automating repetitive digital workflows, which can benefit users with disabilities and lower the barrier to computer use in general. At the same time, stronger GUI agents could be misused to perform unwanted actions, interact with services without proper authorization, or expose sensitive information if deployed without appropriate safeguards. Responsible deployment should therefore include explicit user consent, sandboxing or permission boundaries, audit logs, and privacy-preserving data handling. Because STP relies only on step-level state transitions and does not require task-level instructions, it avoids one source of sensitive information, as task instructions may embed personal context or private details; however, screenshots themselves can still contain private content and require the same careful data handling. This paper studies training methods on existing research benchmarks and does not release a new high-risk dataset or an autonomous deployed system.

7 Conclusion

We introduce State Transition Pretraining (STP) as a scalable complement to trajectory fine-tuning for GUI agents. By jointly optimizing inverse and forward dynamics on step-level tuples (s_t, a_t, s_{t+1}) that can be automatically collected without task annotations, STP consistently improves downstream performance across desktop and mobile benchmarks. Our results further show that the two objectives provide complementary supervision, that performance improves with transition data volume, and that STP remains beneficial as trajectory fine-tuning data scales. Additional analysis reveals that STP is equally effective whether transitions are decomposed from trajectories or collected at the step level, that improving visual representations alone is insufficient for GUI control, and that the model’s language component must be jointly optimized to fully benefit from state-transition supervision. Taken together, these findings position step-level visual state transitions as an underexploited scaling axis for GUI agents, complementary to trajectory fine-tuning and potentially to reinforcement learning, and suggest that future work on large-scale automated transition collection could unlock further gains without incurring the cost of full trajectory annotation.

References

- Anthropic. Developing a computer use model. <https://www.anthropic.com/research/developing-computer-use>, 2024. Accessed April 27, 2026.
- Anthropic. Claude cowork. <https://www.anthropic.com/product/claude-cowork>, 2026. Product page, accessed April 3, 2026.
- Jiaxin Bai, Yue Guo, Yifei Dong, et al. Patchworld: Gradient-free optimization of executable world models. *arXiv preprint arXiv:2605.30880*, 2026.
- Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, et al. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*, 2025a.
- Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibbo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al. Qwen2.5-VL technical report. *arXiv preprint arXiv:2502.13923*, 2025b.
- Andrea Burns, Kate Saenko, and Bryan Plummer. Tell me what’s next: Textual foresight for generic ui representations. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 4590–4611, 2024.
- Wanxia Cao, Chengzhen Duan, Pei Fu, Pengzhi Gao, et al. Xiaomi-gui-0 technical report. *arXiv preprint arXiv:2606.31410*, 2026a.
- Yilin Cao, Yufeng Zhong, Zhixiong Zeng, Liming Zheng, Jing Huang, Haibo Qiu, Peng Shi, Wenji Mao, and Wan Guanglu. MobileDreamer: Generative sketch world model for GUI agent. *arXiv preprint arXiv:2601.04035*, 2026b.
- Yue Cao, Yingyao Wang, Pi Bu, Jingxuan Xing, Wei Jiang, Zekun Zhu, Junpeng Ma, Sashuai Zhou, Tong Lu, Jun Song, Yu Cheng, Yuning Jiang, and Bo Zheng. Androidlens: Long-latency evaluation with nested sub-targets for android gui agents, 2025. URL <https://arxiv.org/abs/2512.21302>.
- Hyungjoo Chae, Namyoun Kim, Kai Tzu-iunn Ong, Minju Gwak, Gwanwoo Song, Jihoon Kim, Sunghwan Kim, Dongha Lee, and Jinyoung Yeo. Web agents with world models: Learning and leveraging environment dynamics in web navigation. *arXiv preprint arXiv:2410.13232*, 2024.
- Shiqi Chen, Tongyao Zhu, Zian Wang, Jinghan Zhang, Kangrui Wang, Siyang Gao, Teng Xiao, Yee Whye Teh, Junxian He, and Manling Li. Internalizing world models via self-play finetuning for agentic rl. *arXiv preprint arXiv:2510.15047*, 2025a.
- Xiaokang Chen, Zhiyu Wu, Xingchao Liu, Zizheng Pan, Wen Liu, Zhenda Xie, Xingkai Yu, and Chong Ruan. Janus-pro: Unified multimodal understanding and generation with data and model scaling. *arXiv preprint arXiv:2501.17811*, 2025b.
- Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Yantao Li, Jianbing Zhang, and Zhiyong Wu. SeeClick: Harnessing GUI grounding for advanced visual GUI agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. arXiv:2401.10935.
- Chaorui Deng, Deyao Zhu, Kunchang Li, Chenhui Gou, Feng Li, Zeyu Wang, Shu Zhong, Weihao Yu, Xiaonan Nie, Ziang Song, Guang Shi, and Haoqi Fan. Emerging properties in unified multimodal pretraining. *arXiv preprint arXiv:2505.14683*, 2025.
- Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web. *Advances in Neural Information Processing Systems*, 36:28091–28114, 2023.
- FAIR CodeGen team, Jade Copet, Quentin Carbonneaux, Gal Cohen, Jonas Gehring, Jacob Kahn, Jannik Kossen, Felix Kreuk, Emily McMilin, Michel Meyer, Yuxiang Wei, et al. CWM: An open-weights llm for research on code generation with world models. *arXiv preprint arXiv:2510.02387*, 2025.

-
- Aarash Feizi, Shravan Nayak, Xiangru Jian, Kevin Qinghong Lin, Kaixin Li, Rabiul Awal, Xing Han Lù, Johan Obando-Ceron, Juan A Rodriguez, Nicolas Chapados, et al. Grounding computer use agents on human demonstrations. *arXiv preprint arXiv:2511.07332*, 2025.
- Longxi Gao, Li Zhang, Pengzhi Gao, Wei Liu, Jian Luan, and Mengwei Xu. GUI-Shift: Enhancing VLM-based GUI agents through self-supervised reinforcement learning. *arXiv preprint arXiv:2505.12493*, 2025.
- Li Gu, Zihuan Jiang, Zhixiang Chi, Huan Liu, Ziqiang Wang, Yuanhao Yu, Glen Berseth, and Yang Wang. Generalization in online reinforcement learning for mobile agents. *arXiv preprint arXiv:2603.07432*, 2026.
- Zhangxuan Gu, Zhengwen Zeng, Zhenyu Xu, Xingran Zhou, Shuheng Shen, Yunfei Liu, Beitong Zhou, Changhua Meng, Tianyu Xia, Weizhi Chen, et al. Ui-venus technical report: Building high-performance ui agents with rft. *arXiv preprint arXiv:2508.10833*, 2025.
- Yiming Guan, Rui Yu, John Zhang, Lu Wang, Chaoyun Zhang, Liqun Li, Bo Qiao, Si Qin, He Huang, Fangkai Yang, et al. Computer-using world model. *arXiv preprint arXiv:2602.17365*, 2026.
- Yifei He, Pranit Chawla, Yaser Souri, Subhojit Som, and Xia Song. WebSTAR: Scalable data synthesis for computer use agents with step-level filtering. *arXiv preprint arXiv:2512.10962*, 2025.
- Yu-Chung Hsiao, Fedir Zubach, Gilles Baechler, Srinivas Sunkara, Victor Cărbune, Jason Lin, Maria Wang, Yun Zhu, and Jindong Chen. Screenqa: Large-scale question-answer pairs over mobile app screenshots. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 9427–9452, 2025.
- Oğuzhan Fatih Kar, Roman Bachmann, Yuting Gong, Anders Boesen Lindbo Larsen, and Afshin Dehghan. Weblica: Scalable and reproducible training environments for visual web agents. *arXiv preprint arXiv:2605.06761*, 2026.
- Woosung Koh, Sungjun Han, Segyu Lee, Se-Young Yun, and Jamin Shin. Generative visual code mobile world models. *arXiv preprint arXiv:2602.01576*, 2026.
- Hanyu Lai, Xiao Liu, Yanxiao Zhao, Han Xu, Hanchen Zhang, Bohao Jing, Yanyu Ren, Shuntian Yao, Yuxiao Dong, and Jie Tang. Computerrl: Scaling end-to-end online reinforcement learning for computer use agents. *arXiv preprint arXiv:2508.14040*, 2025.
- Kaixin Li, Ziyang Meng, Hongzhan Lin, Ziyang Luo, Yuchen Tian, Jing Ma, Zhiyong Huang, and Tat-Seng Chua. Screenspot-pro: Gui grounding for professional high-resolution computer use. In *Proceedings of the 33rd ACM International Conference on Multimedia*, pp. 8778–8786, 2025.
- Wei Li, William Bishop, Alice Li, Chris Rawles, Folawiyo Campbell-Ajala, Divya Tyamagundlu, and Oriana Riva. On the effects of data scale on UI control agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. *arXiv:2406.03679*.
- Zichuan Lin, Feiyu Liu, Yijun Yang, Jiafei Lyu, Yiming Gao, Yicheng Liu, Zhicong Lu, Yangbin Yu, Mingyu Yang, Junyou Li, et al. Ui-voyager: A self-evolving gui agent learning via failed experience. *arXiv preprint arXiv:2603.24533*, 2026.
- Guangyi Liu, Pengxiang Zhao, Gao Wu, Yiwen Yin, Mading Li, et al. Mobileforge: Annotation-free adaptation for mobile gui agents with hierarchical feedback-guided policy optimization. *arXiv preprint arXiv:2606.19930*, 2026.
- Junpeng Liu, Yifan Song, Bill Yuchen Lin, Wai Lam, Graham Neubig, Yuanzhi Li, and Xiang Yue. Visualwebbench: How far have multimodal llms evolved in web page understanding and grounding? *arXiv preprint arXiv:2404.05955*, 2024.
- Zhaoyang Liu, JingJing Xie, Zichen Ding, Zehao Li, Bowen Yang, Zhenyu Wu, Xuehui Wang, Qiushi Sun, Shi Liu, Weiyun Wang, et al. ScaleCUA: Scaling open-source computer use agents with cross-platform data. *arXiv preprint arXiv:2509.15221*, 2025.

-
- Dunjie Lu, Yiheng Xu, Junli Wang, Haoyuan Wu, Xinyuan Wang, Zekun Wang, Junlin Yang, Hongjin Su, Jixuan Chen, Junda Chen, et al. Videoagenttrek: Computer use pretraining from unlabeled videos. *arXiv preprint arXiv:2510.19488*, 2025a.
- Fanbin Lu, Zhisheng Zhong, Shu Liu, Chi-Wing Fu, and Jiaya Jia. Arpo: End-to-end policy optimization for gui agents with experience replay. *arXiv preprint arXiv:2505.16282*, 2025b.
- Quanfeng Lu, Wenqi Shao, Zitao Liu, Lingxiao Du, Fanqing Meng, Boxuan Li, Botong Chen, Siyuan Huang, Kaipeng Zhang, and Ping Luo. GUIOdyssey: A comprehensive dataset for cross-app gui navigation on mobile devices. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 22404–22414, 2025c.
- Zhengxi Lu, Jiabo Ye, Fei Tang, Yongliang Shen, Haiyang Xu, Ziwei Zheng, Weiming Lu, Ming Yan, Fei Huang, Jun Xiao, et al. Ui-s1: Advancing gui automation via semi-online reinforcement learning. *arXiv preprint arXiv:2509.11543*, 2025d.
- Dezhao Luo, Bohan Tang, Kang Li, Georgios Papoudakis, Jifei Song, Shaogang Gong, Jianye Hao, Jun Wang, and Kun Shao. ViMo: A generative visual GUI world model for app agents. *arXiv preprint arXiv:2504.13936*, 2025a.
- Run Luo, Lu Wang, Wanwei He, Longze Chen, Jiaming Li, and Xiaobo Xia. GUI-R1: A generalist R1-style vision-language action model for GUI agents. *arXiv preprint arXiv:2504.10458*, 2025b.
- Yiyang Ma, Xingchao Liu, Xiaokang Chen, Wen Liu, Chengyue Wu, Zhiyu Wu, Zizheng Pan, Zhenda Xie, Haowei Zhang, Xingkai Yu, Liang Zhao, Yisong Wang, Jiaying Liu, and Chong Ruan. JanusFlow: Harmonizing autoregression and rectified flow for unified multimodal understanding and generation. *arXiv preprint arXiv:2411.07975*, 2024.
- Jian Mu, Chaoyun Zhang, Chiming Ni, Lu Wang, Bo Qiao, Kartik Mathur, Qianhui Wu, Yuhang Xie, Xiaojun Ma, Mengyu Zhou, Si Qin, Liqun Li, Yu Kang, Minghua Ma, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. Gui-360: A comprehensive dataset and benchmark for computer-using agents, 2025. URL <https://arxiv.org/abs/2511.04307>.
- Brad A Myers. A brief history of human-computer interaction technology. *interactions*, 5(2):44–54, 1998.
- Feiyue Ni, Yanchu Guan, Yuchong Sun, Dong Wang, Chenyi Zhuang, Jinjie Gu, and Ruihua Song. Rega: Reasoning and grounding decoupled gui navigation agents. In *CCF International Conference on Natural Language Processing and Chinese Computing*, pp. 375–387. Springer, 2025.
- OpenAI. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*, 2024. URL <https://arxiv.org/abs/2410.21276>.
- OpenAI. Introducing operator. <https://openai.com/index/introducing-operator/>, 2025. Product page, accessed April 3, 2026.
- OpenAI. Codex: An agent for almost everything. <https://openai.com/index/codex-for-almost-everything/>, April 2026. Product announcement. Describes Codex desktop app updates including computer use, built-in browser, image generation, memory, and plugins.
- Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, et al. UI-TARS: Pioneering automated GUI interaction with native agents. *arXiv preprint arXiv:2501.12326*, 2025.
- Christopher Rawles, Sarah Clinckemaulle, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, et al. Androidworld: A dynamic benchmarking environment for autonomous agents. *arXiv preprint arXiv:2405.14573*, 2024.
- Luke Rivard, Sun Sun, Hongyu Guo, Wenhui Chen, and Yuntian Deng. NeuralOS: Towards simulating operating systems via neural generative models. *arXiv preprint arXiv:2507.08800*, 2025.

-
- Huawen Shen, Chang Liu, Gengluo Li, Xinlong Wang, Yu Zhou, Can Ma, and Xiangyang Ji. Falcon-ui: Understanding gui before following user instructions. *arXiv preprint arXiv:2412.09362*, 2024.
- Zhouzhou Shen, Xueyu Hu, Xiyun Li, Tianqing Fang, Juncheng Li, and Shengyu Zhang. World-model-augmented web agents with action correction. *arXiv preprint arXiv:2602.15384*, 2026.
- Yucheng Shi, Wenhao Yu, Zaitang Li, Yonglin Wang, Hongming Zhang, Ninghao Liu, Haitao Mi, and Dong Yu. Mobilegui-rl: Advancing mobile gui agent through reinforcement learning in online environment. *arXiv preprint arXiv:2507.05720*, 2025.
- Zihan Su, Hongyang Wei, Kangrui Cen, Yong Wang, Guanhua Chen, Chun Yuan, and Xiangxiang Chu. Generation enhances understanding in unified multimodal models via multi-representation generation. *arXiv preprint arXiv:2601.21406*, 2026.
- Qiushi Sun, Kanzhi Cheng, Zichen Ding, Chuanyang Jin, Yian Wang, Fangzhi Xu, Zhenyu Wu, Chengyou Jia, Liheng Chen, Zhounianze Liu, et al. Os-genesis: Automating gui agent trajectory construction via reverse task synthesis. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 5555–5579, 2025.
- Fei Tang, Zhiqiong Lu, Boxuan Zhang, Weiming Lu, Jun Xiao, Yueting Zhuang, and Yongliang Shen. Clawgui: A unified framework for training, evaluating, and deploying gui agents, 2026a. URL <https://arxiv.org/abs/2604.11784>.
- Zhengyang Tang, Yuxuan Liu, et al. Phoneworld: Scaling phone-use agent environments. *arXiv preprint arXiv:2605.29486*, 2026b.
- Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, SH Cai, Yuan Cao, Y Charles, HS Che, Cheng Chen, Guanduo Chen, et al. Kimi K2.5: Visual agentic intelligence. *arXiv preprint arXiv:2602.02276*, 2026a.
- Venus Team, Changlong Gao, Zhangxuan Gu, Yulin Liu, Xinyu Qiu, Shuheng Shen, Yue Wen, Tianyu Xia, Zhenyu Xu, Zhengwen Zeng, et al. Ui-venus-1.5 technical report. *arXiv preprint arXiv:2602.09082*, 2026b.
- Haoming Wang, Haoyang Zou, Huatong Song, Jiazhan Feng, Junjie Fang, et al. UI-TARS-2 technical report: Advancing GUI agent with multi-turn reinforcement learning. *arXiv preprint arXiv:2509.02544*, 2025a.
- Haonan Wang, Hongfu Liu, Xiangyan Liu, Chao Du, Kenji Kawaguchi, Ye Wang, and Tianyu Pang. Fostering video reasoning via next-event prediction. *arXiv preprint arXiv:2505.22457*, 2025b.
- Xinyuan Wang, Bowen Wang, Dunjie Lu, Junlin Yang, Tianbao Xie, Junli Wang, Jiaqi Deng, Xiaole Guo, Yiheng Xu, Chen Henry Wu, Zhennan Shen, Zhuokai Li, Ryan Li, Xiaochuan Li, Junda Chen, Boyuan Zheng, Peihang Li, Fangyu Lei, Ruisheng Cao, Yeqiao Fu, Dongchan Shin, Martin Shin, Jiarui Hu, Yuyan Wang, Jixuan Chen, Yuxiao Ye, Danyang Zhang, Dikang Du, Hao Hu, Huarong Chen, Zaida Zhou, Haotian Yao, Ziwei Chen, Qizheng Gu, Yipu Wang, Heng Wang, Diyi Yang, Victor Zhong, Flood Sung, Y. Charles, Zhilin Yang, and Tao Yu. OpenCUA: Open foundations for computer-use agents. <https://opencua.xlang.ai/>, 2025c. AgentNet dataset and AgentNetBench described on project page.
- Di Wu, Ruitao Hao, Haiyang Wang, Shuzhe Wu, Han Xiao, et al. Mobilegym: A verifiable and highly parallel simulation platform for mobile gui agent research. *arXiv preprint arXiv:2605.26114*, 2026a.
- Jialong Wu, Xiaoying Zhang, Hongyi Yuan, Xiangcheng Zhang, Tianhao Huang, Changjing He, Chaoyi Deng, Renrui Zhang, Youbin Wu, and Mingsheng Long. Visual generation unlocks human-like reasoning through multimodal world models. *arXiv preprint arXiv:2601.19834*, 2026b.
- Mengzhou Wu, Yuzhe Guo, Yuan Cao, Haochuan Lu, Songhe Zhu, Pingzhe Qu, Xin Chen, Kang Qin, Zhongpu Wang, Xiaode Zhang, et al. Ui-oceanus: Scaling gui agents with synthetic environmental dynamics. *arXiv preprint arXiv:2604.02345*, 2026c.
- Size Wu, Zhonghua Wu, Zerui Gong, Qingyi Tao, Sheng Jin, Qinyue Li, Wei Li, and Chen Change Loy. Openuni: A simple baseline for unified multimodal understanding and generation. *arXiv preprint arXiv:2505.23661*, 2025a.

-
- Size Wu, Wenwei Zhang, Lumin Xu, Sheng Jin, Zhonghua Wu, Qingyi Tao, Wentao Liu, Wei Li, and Chen Change Loy. Harmonizing visual representations for unified multimodal understanding and generation. *arXiv preprint arXiv:2503.21979*, 2025b.
- Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, et al. Os-atlas: A foundation action model for generalist gui agents. *arXiv preprint arXiv:2410.23218*, 2024.
- Jinheng Xie, Weijia Mao, Zechen Bai, David Junhao Zhang, Weihao Wang, Kevin Qinghong Lin, Yuchao Gu, Zhijie Chen, Zhenheng Yang, and Mike Zheng Shou. Show-o: One single transformer to unify multimodal understanding and generation. In *ICLR*, 2025.
- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2404.07972.
- Weimin Xiong, Shuhao Gu, Bowen Ye, Zihao Yue, Lei Li, Feifan Song, Sujian Li, and Hao Tian. Video2gui: Synthesizing large-scale interaction trajectories for generalized gui agent pretraining. *arXiv preprint arXiv:2605.14747*, 2026.
- Haiyang Xu, Xi Zhang, Haowei Liu, Junyang Wang, Zhaozai Zhu, Shengjie Zhou, Xuhao Hu, Feiyu Gao, Junjie Cao, Zihua Wang, Zhiyuan Chen, Jitong Liao, Qi Zheng, Jiahui Zeng, Ze Xu, Shuai Bai, Junyang Lin, Jingren Zhou, and Ming Yan. Mobile-agent-v3.5: Multi-platform fundamental gui agents, 2026a. URL <https://arxiv.org/abs/2602.16855>.
- Weikai Xu, Kun Huang, Yu Feng, Jiaying Li, Yuhao Chen, et al. How mobile world model guides gui agents? *arXiv preprint arXiv:2605.10347*, 2026b.
- Yiheng Xu, Zekun Wang, Junli Wang, Dunjie Lu, Tianbao Xie, Amrita Saha, Doyen Sahoo, Tao Yu, and Caiming Xiong. Aguis: Unified pure vision agents for autonomous gui interaction. *arXiv preprint arXiv:2412.04454*, 2024.
- Taofeng Xue, Chong Peng, Mianqiu Huang, Linsen Guo, Tiancheng Han, Haozhe Wang, Jianing Wang, Xiaocheng Zhang, Xin Yang, Dengchang Zhao, Jinrui Ding, Xiandi Ma, Yuchen Xie, Peng Pei, Xunliang Cai, and Xipeng Qiu. Evocua: Evolving computer use agents via learning from scalable synthetic experience, 2026. URL <https://arxiv.org/abs/2601.15876>.
- Yuhao Yang, Yue Wang, Dongxu Li, Ziyang Luo, Bei Chen, Chao Huang, and Junnan Li. Aria-UI: Visual grounding for GUI instructions. *arXiv preprint arXiv:2412.16256*, 2024.
- Yuhao Yang, Zhen Yang, Zi-Yi Dou, Anh Nguyen, Keen You, Omar Attia, Andrew Szot, Michael Feng, Ram Ramrakhya, Alexander Toshev, Chao Huang, Yinfei Yang, and Zhe Gan. Ultracua: A foundation model for computer use agents with hybrid action, 2025. URL <https://arxiv.org/abs/2510.17790>.
- Zhenquan Yao, Zitong Huang, Yihan Zeng, Jianhua Han, Hang Xu, Chun-Mei Feng, Jianwei Ma, and Wangmeng Zuo. Cgl: Advancing continual gui learning via reinforcement fine-tuning. *arXiv preprint arXiv:2603.02951*, 2026.
- Jiabo Ye, Xi Zhang, Haiyang Xu, Haowei Liu, Junyang Wang, Zhaoqing Zhu, Ziwei Zheng, Feiyu Gao, Junjie Cao, Zhengxi Lu, et al. Mobile-agent-v3: Fundamental agents for gui automation. *arXiv preprint arXiv:2508.15144*, 2025.
- Xue Yu, Bo Yuan, Pengjian Yang, Kailin Zhao, Hong Hu, and Junlan Feng. Seerguard: A safety framework for mobile gui agents via world model prediction. *arXiv preprint arXiv:2607.15550*, 2026.
- Kai Zhang, Xiangchao Chen, Bo Liu, Tianci Xue, Zeyi Liao, Zhihan Liu, Xiyao Wang, Yuting Ning, Zhaorun Chen, Xiaohan Fu, et al. Agent learning via early experience. *arXiv preprint arXiv:2510.08558*, 2025a.

-
- Wenyao Zhang, Bozhou Zhang, Zekun Qi, Wenjun Zeng, Xin Jin, and Li Zhang. Disentangled robot learning via separate forward and inverse dynamics pretraining. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=DdrSHWobR1>.
- Zhong Zhang, Yaxi Lu, Yikun Fu, Yupeng Huo, Shenzhi Yang, Yesai Wu, Han Si, Xin Cong, Haotian Chen, Yankai Lin, et al. Agentcpm-gui: Building mobile-use agents with reinforcement fine-tuning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 155–180, 2025b.
- Shanshan Zhao, Xinjie Zhang, Jintao Guo, Jiakui Hu, Lunhao Duan, Minghao Fu, Yong Xien Chng, Guo-Hua Wang, Qing-Guo Chen, Zhao Xu, et al. Unified multimodal understanding and generation models: Advances, challenges, and opportunities. *arXiv preprint arXiv:2505.02567*, 2025.
- Yuhao Zheng, Li’an Zhong, Yi Wang, Rui Dai, Kaikui Liu, Xiangxiang Chu, Linyuan Lv, Philip Torr, and Kevin Qinghong Lin. Code2world: A gui world model via renderable code generation. *arXiv preprint arXiv:2602.09856*, 2026.
- Chunting Zhou, Lili Yu, Arun Babu, Kushal Tirumala, Michihiro Yasunaga, Leonid Shamis, Jacob Kahn, Xuezhe Ma, Luke Zettlemoyer, and Omer Levy. Transfusion: Predict the next token and diffuse images with one multi-modal model. In *ICLR*, 2025a.
- Hanzhang Zhou, Xu Zhang, Panrong Tong, Jianan Zhang, Liangyu Chen, Quyu Kong, Chenglin Cai, Chen Liu, Yue Wang, Jingren Zhou, et al. Mai-ui technical report: Real-world centric foundation gui agents. *arXiv preprint arXiv:2512.22047*, 2025b.
- Yuxin Zuo, Zikai Xiao, Li Sheng, Fei Huang, Jianhong Tu, et al. Qwen-agentworld: Language world models for general agents. *arXiv preprint arXiv:2606.24597*, 2026.

A Trajectory-to-Sample Conversion Examples

This appendix illustrates how an annotated trajectory is converted into training samples, using AgentNet as an example. An N -step trajectory $\tau = (g, s_1, a_1, s_2, a_2, \dots)$ yields N trajectory fine-tuning samples (one per step) and $N - 1$ transition tuples (s_t, a_t, s_{t+1}) , each of which instantiates one inverse dynamics sample and one forward dynamics sample. The abridged formats are shown below, where `<image>` denotes a screenshot input and the full action-format specification in the system prompt is omitted for brevity.

Trajectory fine-tuning sample (step S). The input contains the system prompt, the textual actions of all previous steps (past screenshots are excluded, as described in Section 3.1), the current screenshot s_S , and the task instruction; the target is the action of the current step.

```
[System]    You are a GUI agent. You are given a task and a screenshot
            of the screen. You need to perform a series of pyautogui
            actions to complete the task. <action-format specification>

[Assistant] # Step 1
            ## Action:
            Open the document "report.docx" in LibreOffice Writer.
            ...

[Assistant] # Step S-1
            ## Action:
            Click the "File" menu in the top-left corner.

[User]      <image: current screenshot s_S>
            # Task Instruction:
            Export the opened document as PDF and save it to Desktop.

            Please generate the next move according to the screenshot,
```

task instruction and previous steps (if provided).

[Target] # Step S
 ## Action:
 Click the "Export as PDF" button in the dialog.
 ## Code:
 '''python
 pyautogui.click(x=0.7318, y=0.4642)
 '''

Inverse dynamics sample. The input contains the two consecutive screenshots; the target is the action that caused the transition.

[User] You are a helpful assistant that infers actions from
 screen transitions.

 Previous screen: <image: s_t>
 Current screen: <image: s_{t+1}>

 Action format: <action-format specification>

 What action was performed?

[Target] ## Action:
 Click the "Export as PDF" button in the dialog.
 ## Code:
 '''python
 pyautogui.click(x=0.7318, y=0.4642)
 '''

Forward dynamics sample. The input contains the current screenshot and the action; the target is the next screenshot, supervised by the flow matching objective.

[User] You are a helpful assistant that predicts the next screen
 state. Given the current screen, perform the move:

 ## Action:
 Click the "Export as PDF" button in the dialog.
 ## Code:
 '''python
 pyautogui.click(x=0.7318, y=0.4642)
 '''

 What is the next screen?
 <image: s_t>

[Target] <image: s_{t+1}>